Web and Data Engineering

Kapitel 01: Organisation und Einleitung

Prof. Dr. Michael Granitzer

Ziel dieser Vorlesung

Moderne Web-Systeme verlangen systematisches Engineering - über Architektur, Teams, Daten und AI hinweg. Diese Einheit klärt, **warum** das so ist und **was** Sie in diesem Kurs dafür lernen werden.

Leitfragen

- Warum ist das Web mehr als ein Informationskanal?
- Was macht Web-Systeme komplex - technisch und organisatorisch?
- Warum reicht Coding allein nicht - und was tritt an seine Stelle?
- Wie verändert AI die Rolle von Web Engineers?
- Warum gehört die Datenperspektive dazu?

Das Web als Infrastruktur

Leitfrage: Warum ist das Web mehr als nur ein Informationskanal, nämlich eine technische und gesellschaftliche Kerninfrastruktur?

Das Web als Infrastruktur

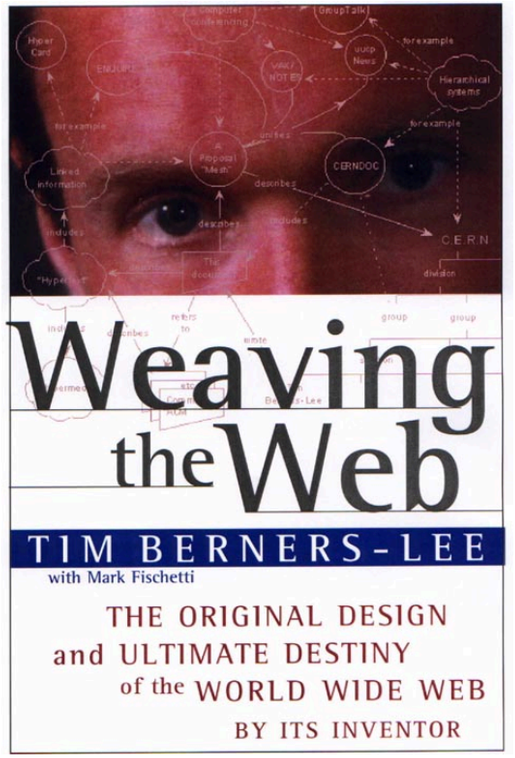
Das World Wide Web ist heute weit mehr als eine Sammlung von Webseiten. Es ist die zentrale Infrastruktur für digitale Kommunikation, Plattformen, datengetriebene Anwendungen - und zunehmend auch für autonome AI-Systeme.

Diese Infrastruktur hat sich in mehreren Paradigmenwechseln entwickelt. Jeder Schritt hat neue Engineering-Probleme erzeugt, die diesen Kurs motivieren.

Von der Vision zum Netz (Web 1.0)

> **Problem:** Wie macht man Wissen maschinengestützt zugänglich?

- 1969: `ARPANET` zeigt, dass verteilte Netze praktisch funktionieren
- 1989: Tim Berners-Lee schlägt am CERN das `World Wide Web` vor
- 1991: das erste öffentliche Webangebot geht online
  - statische Dokumente mit **Hyperlinks**
  - **HTTP** als Protokoll
  - **HTML** als Auszeichnungssprache
  - **URI** als Adressierungsschema



Offene Protokolle und verteilte Infrastruktur werden zum Fundament - aber das Web liefert zunächst nur **statische Dokumente**.

Von Dokumenten zu Anwendungen (Web 1.0 -> Web 2.0)

> **Problem:** Wie wird das Web interaktiv und kommerziell nutzbar?

- 1995: `JavaScript` und serverseitige Technologien (`PHP`, `CGI`) ermöglichen dynamische Seiten
- 1995-2000: `Dotcom-Boom` - E-Commerce (`Amazon`, `eBay`), Webmail, Online-Banking
- 2001: der Dotcom-Crash - es überlebten nur Systeme, die technisch skalierbar gebaut wurden
- 2001: Das Web 2.0 wird propagiert:
  - Benutzer werden zu Produzenten von Inhalten (Prosumer) und neue Geschäftsmodelle entstehen
  - Daten und Netzwerkeffekte werden zum zentralen Wettbewerbsvorteil
  - Perpetual Beta - Software wird kontinuierlich weiterentwickelt und verbessert
  - Rich User Experience - Rich Client Applications

Interaktivität erzeugt **Zustand, Sicherheitsfragen und Serverlogik** - das Web braucht plötzlich Software-Architektur.



Von Anwendungen zu Plattformen (Web 2.0)

> **Problem:** Wie wird das Web zur universellen Betriebsplattform?

- 2006: `AWS` startet - Cloud-Infrastruktur wird on-demand verfügbar
- 2007: `iPhone` macht das Web allgegenwärtig; Responsive Design wird Pflicht
- 2010er: `Microservices` + `Kubernetes` - kontinuierliches Deployment zum Standard; APIs als Integrationsschicht
  - HTML 5, CSS 3, JavaScript ES6+ - Rich Client Applications werden zum Standard
  - AJAX, JSON, REST APIs - Daten werden zum zentralen Element
  - Unterschiede zu Desktop Programmen reduziert sich

Verteilte Systeme, Skalierung und Teamkoordination werden zu **zentralen Engineering-Problemen**. Das Web ist nicht mehr Anwendung, sondern **Betriebsplattform**.

Von Plattformen zu datengetriebenen Systemen (Web 2.0 -> Web 3.0)

> **Problem:** Wie nutzt man die Daten, die Web-Systeme erzeugen?

- Plattformen erzeugen kontinuierlich Daten - Klicks, Käufe, Suchanfragen
- Personalisierung (`Netflix`, `Spotify`) wird vom Zusatzfeature zur **Kernfunktion**
- `Data Pipelines` entstehen als eigenstaendige Infrastruktur; neue Rollen wie `Data Engineer`
- Maschinelles Lernen und diskriminative KI Algorithmen werden integraler Bestandteil von Web-Systemen
  - z.B. Empfehlungsdienste, Suchmaschinen, etc.
  - Rankings und Algorithmic Bias
  - Filter Bubbles

Data Engineering wird integraler Bestandteil von Web-Systemen - genau deshalb verbindet dieser Kurs beide Perspektiven.

Vom menschlichen Nutzer zum AI-Agenten (Web 3.0)

Problem: Wer nutzt das Web in Zukunft - und wie?

- AI-Agenten nutzen zunehmend **Web-basierte Tools**: Suchmaschinen, Webseiten-Inhalte, REST-APIs
- Protokolle wie das `Model Context Protocol (MCP)` standardisieren Nutzung durch Agenten
- Agenten **orchestrieren Ketten von Web-Interaktionen** autonom
- das Web wird zur **Ausführungsumgebung für autonome Systeme**, nicht nur für menschliche Nutzer

Agent-Aktion	Web-Technologie
Informationssuche	Web Search APIs
Seiteninhalt lesen	HTTP + HTML Parsing
Daten abrufen	REST / GraphQL APIs
Aktionen ausführen	Web APIs mit Auth
Tool-Integration	MCP-Server über HTTP

Konsequenz (vermutlich): Web-APIs und -Dienste müssen nicht nur für Menschen, sondern auch für **maschinelle Konsumenten** robust, dokumentiert und sicher sein. Das

Die Paradigmenwechsel im Überblick



Jeder Paradigmenwechsel erzeugt neue Engineering-Probleme — und neue Anforderungen an Web-Systeme.

Jeder Paradigmenwechsel im Web hat neue Engineering-Probleme erzeugt. Mit AI-Agenten steht der nächste bevor: Das Web wird nicht nur Plattform für Menschen, sondern Ausführungsumgebung für autonome Systeme. Die Relevanz des Webs liegt darin, dass hier offene Protokolle, Anwendungslogik und Datenströme zu einem universellen Ausführungsraum zusammenkommen - für menschliche und maschinelle Akteure gleichermaßen.

Was Web-Systeme komplex macht

Leitfrage: Was genau macht Web-Systeme so komplex, dass einfaches Programmieren nicht ausreicht?

Definition Web Engineering

> *Web Engineering ist die Anwendung systematischer und quantifizierbarer Ansätze für Anforderungsbeschreibung, Entwurf, Implementierung, Test, Betrieb und Wartung qualitativ hochwertiger Web-Anwendungen.*

Kerngedanke

Web Engineering beschreibt sowohl eine praktische Ingenieurstätigkeit als auch eine wissenschaftliche Disziplin.

Was sind Web-Anwendungen?

Typische Merkmale

- Nutzung über Web-Protokolle und Browser
- verteilte Komponenten
- Kombination aus UI, Logik und Datenhaltung
- häufig kontinuierliche Weiterentwicklung

Typische Unterschiede

- informationsorientiert vs. transaktionsorientiert
- einfache Sites vs. komplexe Plattformen
- monolithisch vs. verteilt
- rein webbasiert vs. stark datengetrieben

Skala und Heterogenität des Webs

Das Web heute: Skala und Heterogenität

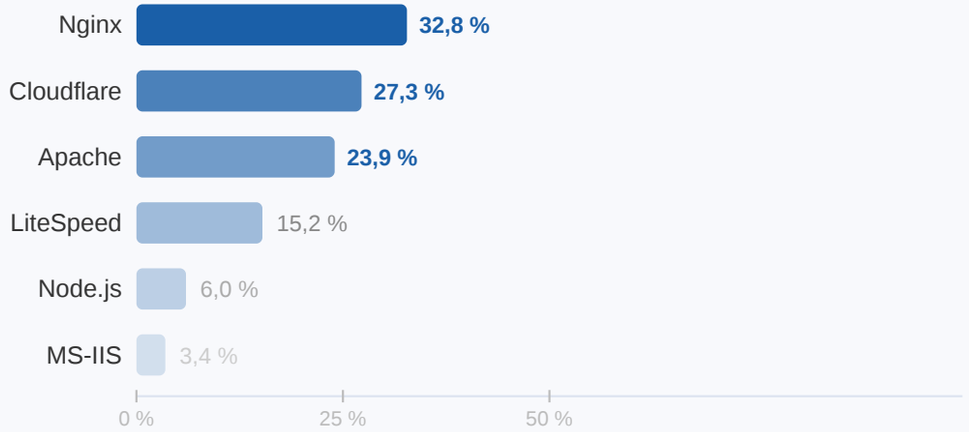
≈ 1,1 Mrd.
Websites weltweit
Internet Live Stats, 2024

16,9 Mio.
analysierte Sites (HTTP Archive)
Web Almanac 2024

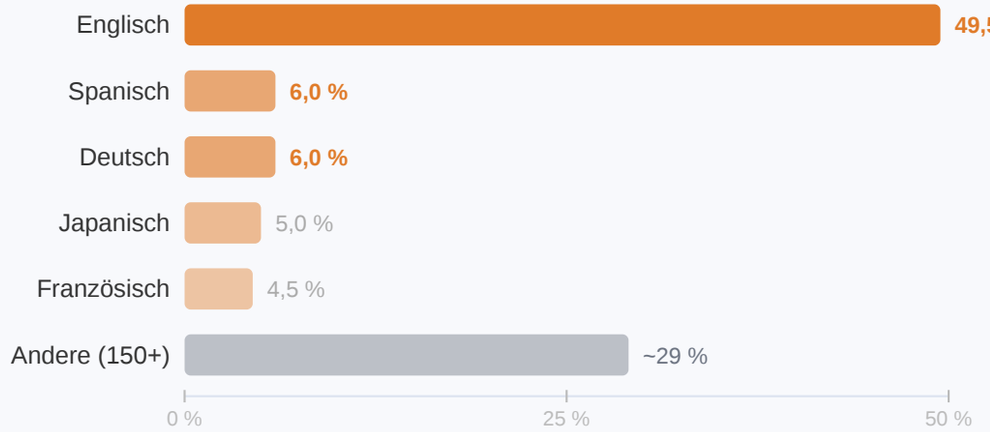
2,6 MB
Median Desktop (71 HTTP-Requests)
Web Almanac 2024

249 Plattformen
erkannte CMS · 51 % aller Sites
Web Almanac 2024

Web-Server Technologien (W3Techs 2024)



Sprachen der Web-Inhalte (W3Techs 2024)



Quellen: HTTP Archive Web Almanac 2024 (Viscomi et al., Nov. 2024) · W3Techs 2024 · Internet Live Stats 2024. Daten beziehen sich auf öffentlich erreichbare Websites.

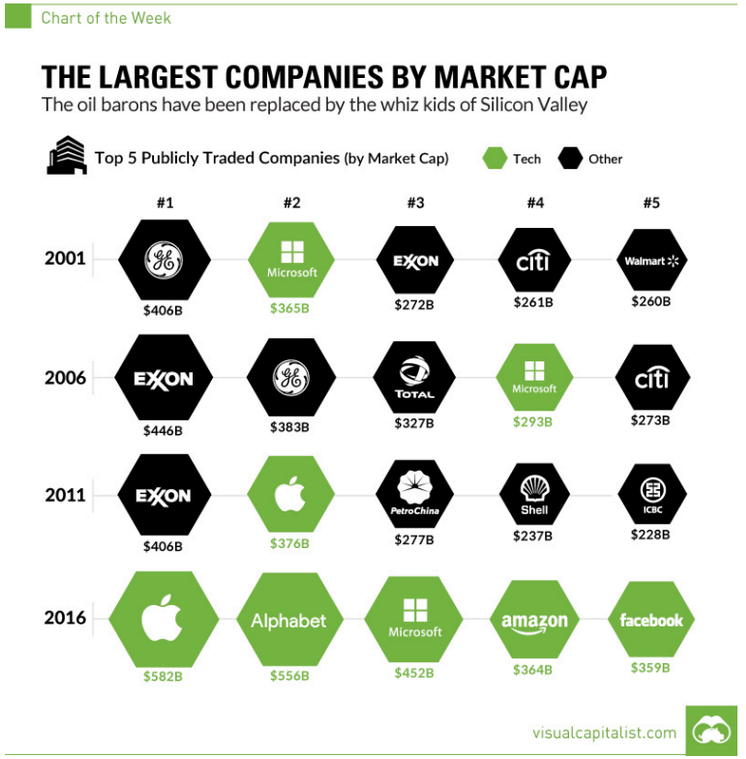
Source: Eigene Visualisierung auf Basis von HTTP Archive Web Almanac 2024 (Viscomi et al.) und W3Techs 2024. Daten: öffentlich erreichbare Websites, Erhebungszeitraum Juni–November 2024.

Das Web ist riesig und hochgradig heterogen: Es umfasst mehr als eine Milliarde Sites, nutzt hunderte verschiedener Technologien und bedient Inhalte in über 150 Sprachen.

Diese Heterogenität ist eine der zentralen Ursachen für die Komplexität von Web-Systemen.

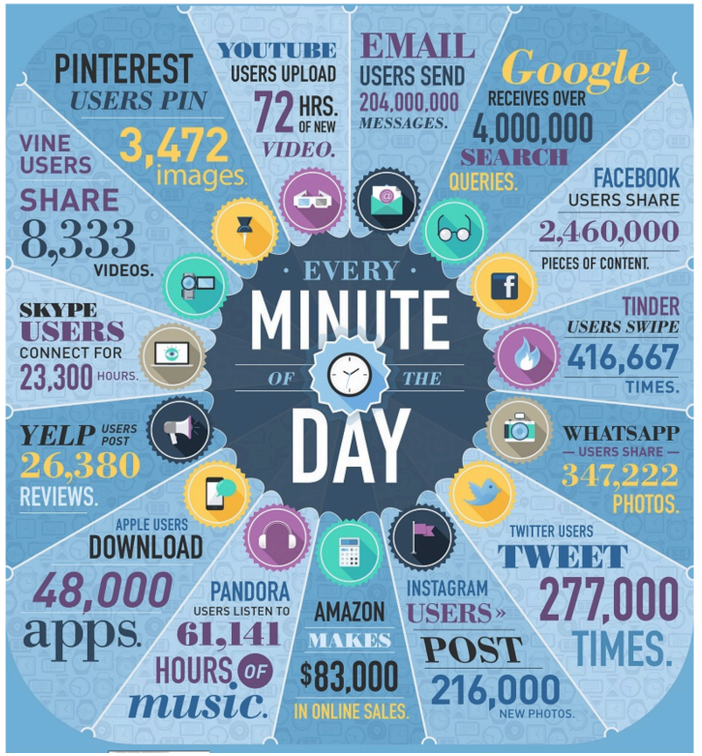
Plattformökonomie

Reichweite und Marktwert digitaler Plattformen entstehen über Web- und Dateninfrastrukturen. Wertschöpfung hängt oft daran, Datenflüsse systematisch zu erfassen und zu nutzen.



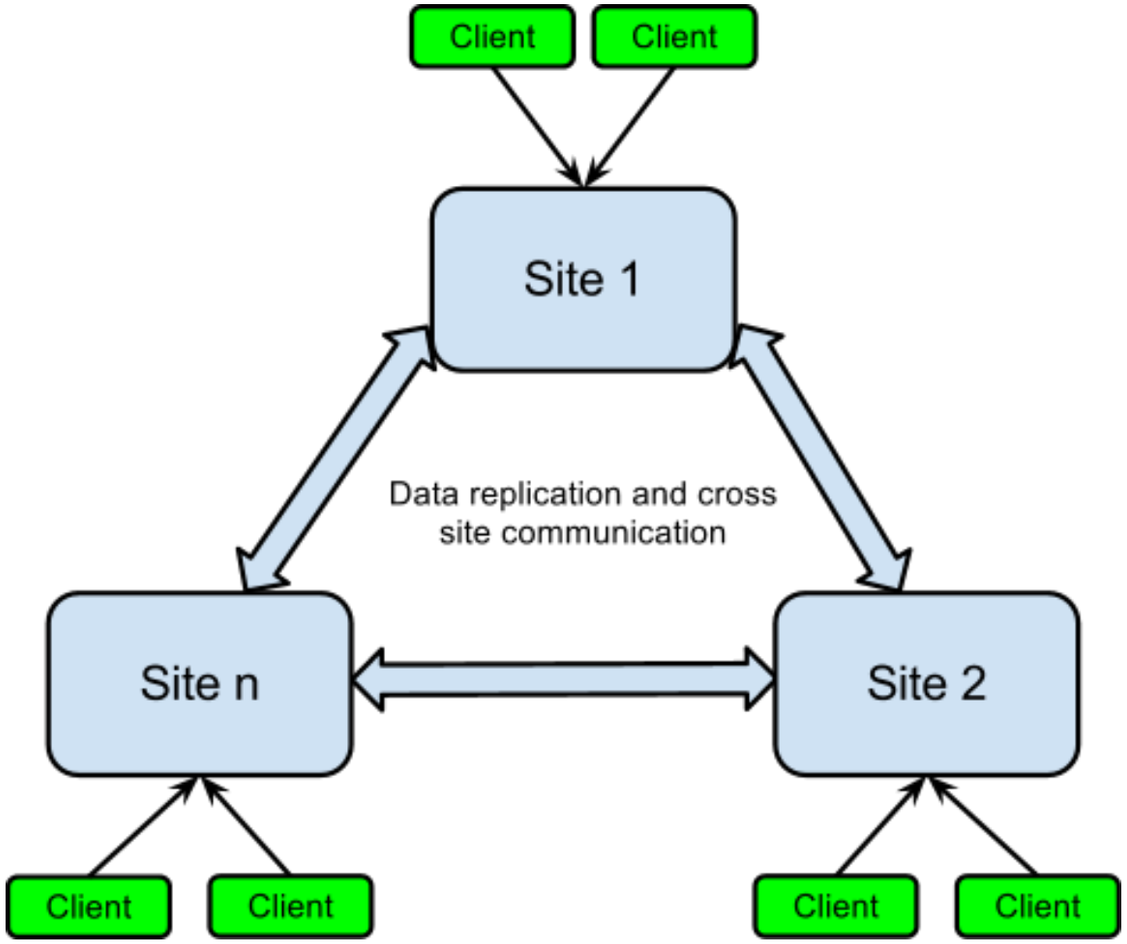
Daten als wirtschaftlicher Hebel

Digitale Plattformen wachsen nicht primär durch bessere Technik, sondern durch systematische Nutzung von Daten. Wer Datenflüsse besser versteht, kann Produkte schneller anpassen, Nutzer gezielter bedienen und neue Geschäftsmodelle erschließen.



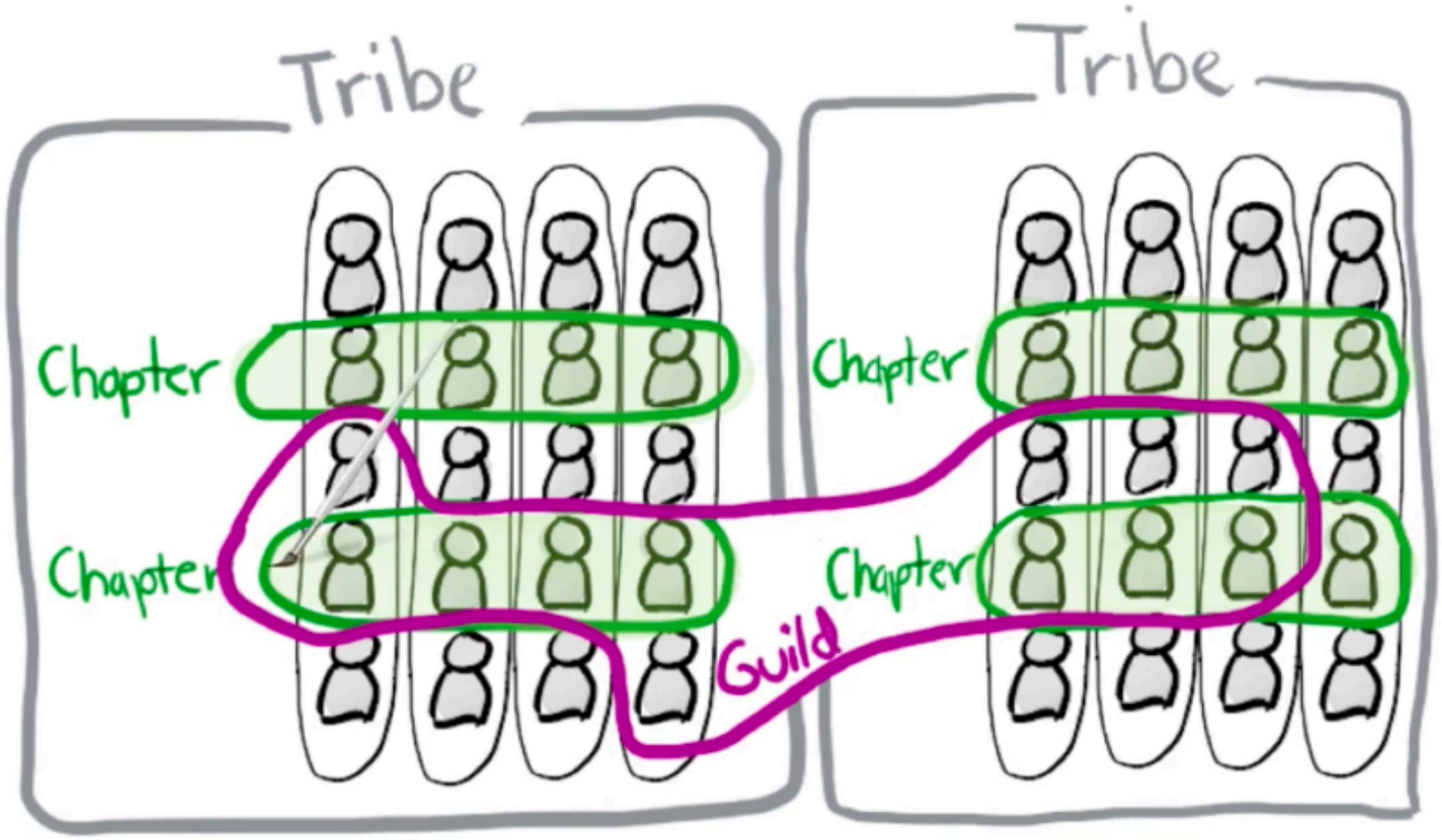
Backend-Komplexität am Beispiel Spotify

Moderne Web-Produkte bestehen aus vielen gekoppelten Diensten. Skalierung, Deployment und Betrieb werden zur Architekturfrage.



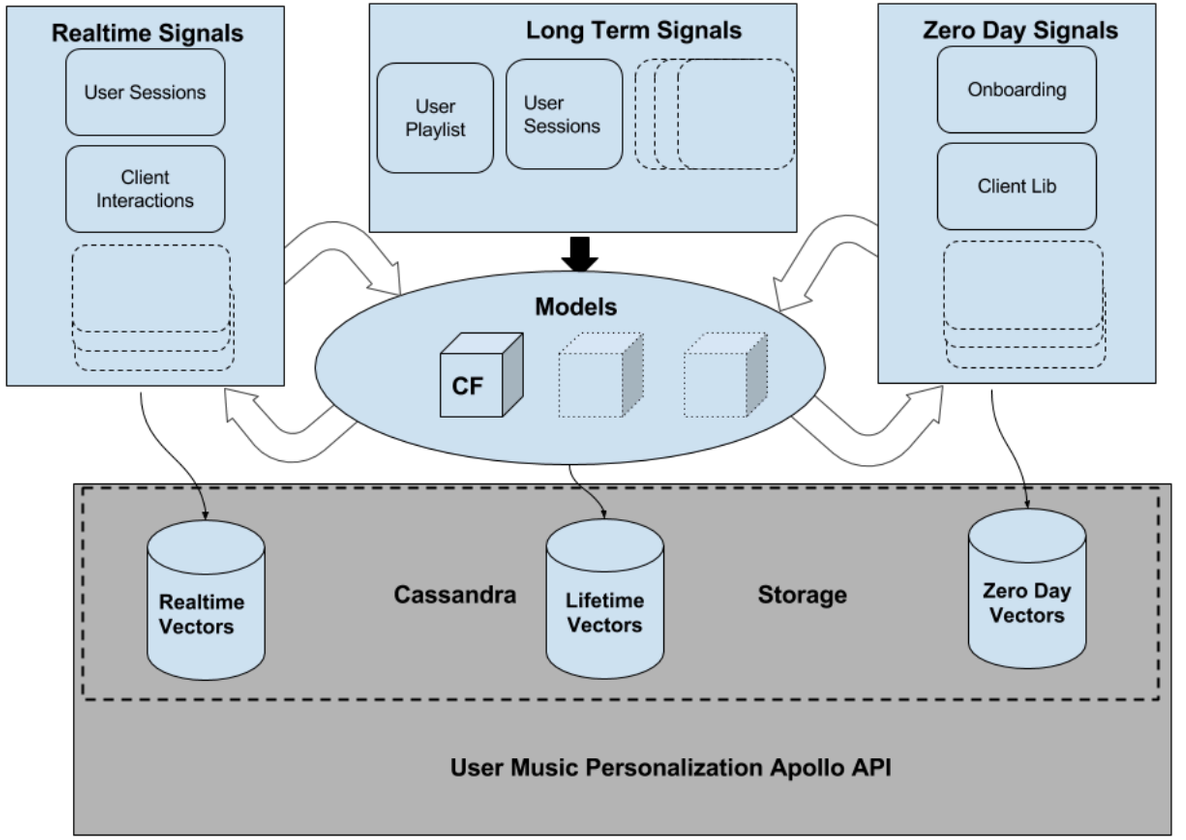
Teamkomplexität am Beispiel Spotify

Technische Komplexitaet trifft auf verteilte Teamstrukturen. Architektur wird auch zum Mittel der Koordination.



AI-Integration am Beispiel Spotify

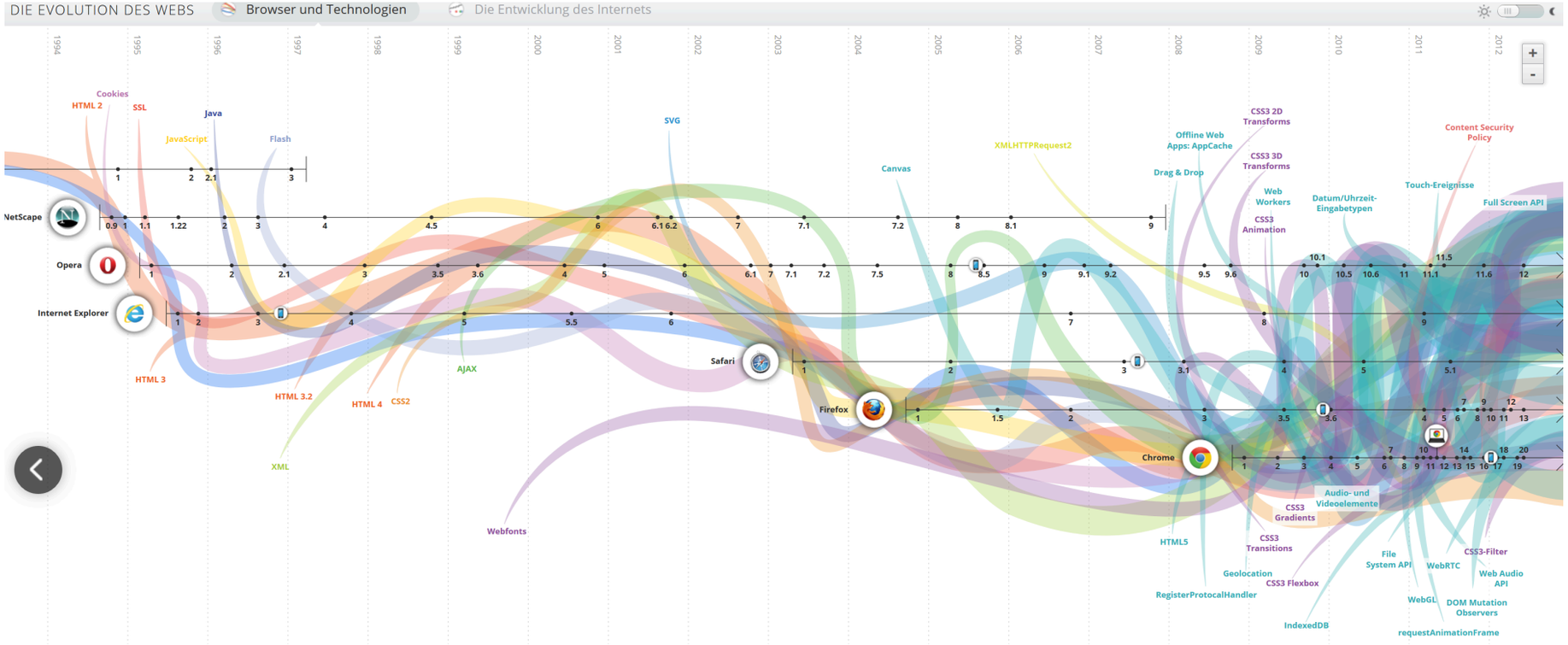
Personalisierung ist kein Zusatzfeature, sondern Teil der Systemarchitektur. Datenpipelines, Modelle und Auslieferung müssen mit Web-Produktlogik zusammenspielen. Damit verschiebt sich Komplexität von der UI in Infrastruktur, Daten und Betrieb.



Komplexität ist der Normalzustand

Technische Dimension

- Frontend, Backend und Persistenz ändern sich nicht unabhängig
- Sicherheit, Verfügbarkeit und Performance sind Querschnittsthemen
- Standards und Schnittstellen entscheiden über Interoperabilität



Organisatorische Dimension

- Teams arbeiten parallel an gekoppelten Teilsystemen
- Änderungen müssen trotz laufendem Betrieb möglich bleiben
- Qualität entsteht über Prozesse, nicht nur über einzelne Code-Artefakte

> Komplexität in Web-Systemen entsteht aus dem Zusammenspiel von Schichten, verteilten Teams und kontinuierlicher Evolution. Diese Komplexität ist kein Ausnahmezustand - sie ist der Normalfall reifer Web-Plattformen.

Vom Coding zum Engineering

Leitfrage: Wenn Web-Systeme so komplex sind - was braucht es über das reine Programmieren hinaus?

Von der Komplexität zur Methode

Was Coding allein nicht löst

- ein einzelnes Feature berührt oft mehrere Schichten gleichzeitig
- Sicherheitslücken, Performanceprobleme und Inkompatibilitäten entstehen zwischen Komponenten
- ohne gemeinsame Methodik entstehen Widersprüche zwischen parallelen Teams

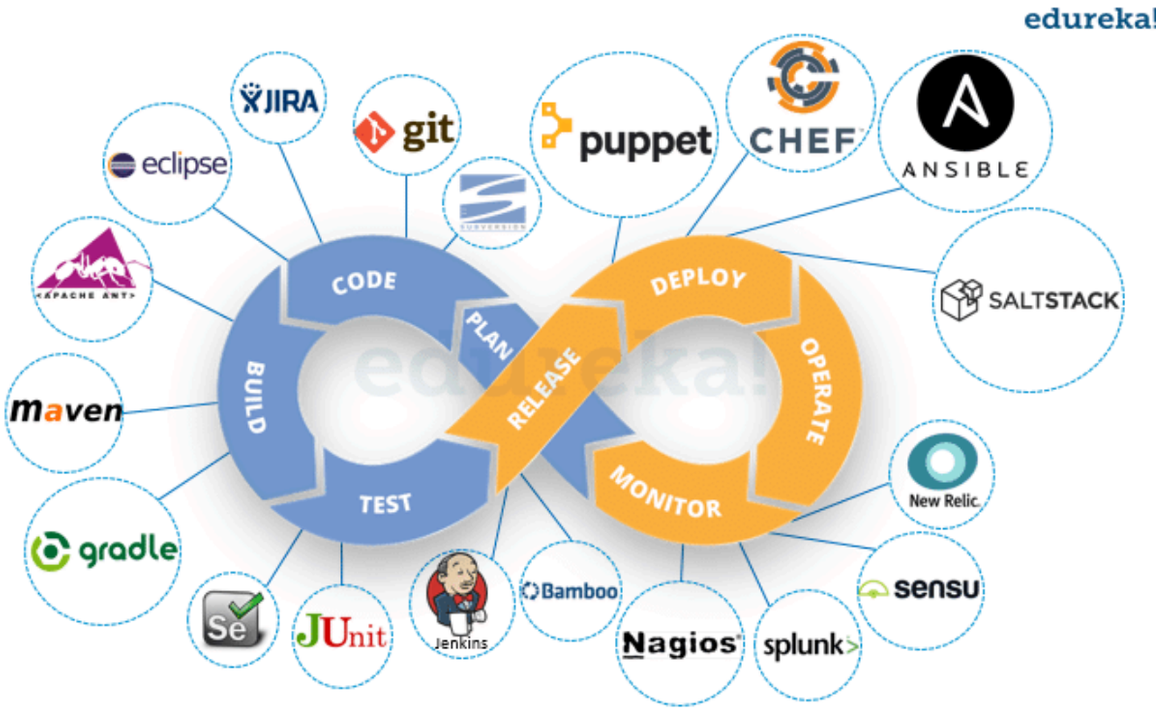
Was Engineering ergänzt

- explizite Anforderungen statt impliziter Annahmen
- bewusste Architekturentscheidungen statt zufällig gewachsener Struktur
- definierte Prozesse für Test, Deployment und Wartung
- Qualitätskriterien, die vor der Implementierung feststehen

Web Engineering als Entwicklungsprozess

Entwicklungszyklus

- Anforderungen, Entwurf, Implementierung und Betrieb müssen zusammen gedacht werden
- Web Engineering ist prozessorientiert, nicht nur technologieorientiert
- Entwicklung, Deployment und Wartung bilden einen zusammenhängenden Lebenszyklus
- Automatisierung, Automatisierung, Automatisierung



Der Lebenszyklus einer modernen Web-Anwendung ist zirkulär: Durch "Observability" (Monitoring, Logging) fließen Daten aus dem Betrieb direkt zurück in die Anforderungsphase. Automatisierung (CI/CD) ist hier kein Komfortmerkmal, sondern die

Standards als Bedingung für Skalierung

Rolle von Standards



- skalierbare Web-Systeme brauchen gemeinsame technische Grundlagen
- Standards schaffen Interoperabilität zwischen Browsern, Servern und Werkzeugen
- Institutionen wie das W3C treiben diese Entwicklung organisatorisch voran

Konsequenz: Standards wie HTML, CSS und HTTP sichern die "Permissionless Innovation" und Interoperabilität. Sie verhindern ein fragmentiertes Web proprietärer Technologien und ermöglichen es erst, dass verteilte Systeme weltweit zusammenarbeiten können. :::

> Web Engineering wird dort notwendig, wo verteilte Technik, Daten, Betrieb und Teamkoordination nicht mehr sauber durch isoliertes Coding beherrschbar sind. Methoden, Prozesse und Standards machen Komplexität handhabbar.

Web Engineering im Zeitalter von AI

Leitfrage: Wenn AI immer mehr Implementierungsarbeit übernimmt, welche Fähigkeiten werden für Web Engineers wichtiger statt weniger wichtig?

Ausgangsthese

Die Routine des Codings wird zunehmend delegierbar. Dadurch wird die eigentliche Engineering-Arbeit wichtiger: Problemzuschnitt, Architektur, Qualitätskriterien, Risikobewertung und Aufsicht über AI-generierte Artefakte.

Evidenz: AI übernimmt bereits spürbare Teile der Implementierung

Produktivität und Delegation

- Microsoft berichtet im Juni 2025 aus drei Feldexperimenten mit 4.867 Entwicklerinnen und Entwicklern im Mittel von +26.08% erledigten Aufgaben mit AI-Unterstützung.
- Anthropic analysierte im April 2025 500.000 coding-bezogene Interaktionen; bei Claude Code wurden 79% als stärker automatisierte Nutzung klassifiziert.

Wo die Automatisierung zuerst greift

- In derselben Anthropic-Analyse dominierten Websprachen wie JavaScript, TypeScript, HTML und CSS.
- User-facing App- und UI/UX-Aufgaben gehörten zu den häufigsten Anwendungsfällen.
- Daraus lässt sich plausibel ableiten, dass gerade einfache Fullstack- und UI-nahe Implementierung früh delegierbar wird.

Von Layer-Spezialisierung zu breiterer Verantwortung

Vor-AI

- viele Entwickler arbeiteten tiefer in einer Schicht des Stacks
- Expertise war oft an ein bestimmtes Framework oder Systemsegment gebunden
- Übergaben zwischen Frontend, Backend, Datenbank und Betrieb waren alltäglich

Mit AI-Unterstützung

- ein einzelner Entwickler kann heute deutlich leichter über mehrere Schichten hinweg liefern
- Anthropic beschreibt diese Entwicklung explizit als "becoming more full-stack"
- damit wächst auch die Verantwortung für das Gesamtsystem statt nur für ein kleines Teilgebiet

Warum konzeptionelles Verständnis wichtiger wird

Wichtiger als früher

- Systemgrenzen und Schnittstellen verstehen
- Anforderungen in überprüfbare Teilprobleme zerlegen
- Architektur- und Trade-off-Entscheidungen treffen
- Korrektheits- und Qualitätskriterien explizit machen

Unwichtiger als früher

- jedes Detail eines Nischentools manuell auswendig beherrschen
- Boilerplate und Standardintegration selbst schreiben
- Implementierungswissen mit Engineering-Verantwortung verwechseln

Die Aufsicht wird zur Kernaufgabe

Warum Supervision nötig bleibt

- AI kann schnell viel Code erzeugen, aber nicht zuverlässig den fachlichen Kontext ersetzen
- reale Entwicklung enthält Mehrdeutigkeit, implizites Domänenwissen und versteckte Randbedingungen
- Fehler in Architektur, Sicherheit oder Datenflüssen sind teurer als Fehler in Syntax

Befund aus Studien

- Anthropic berichtet, dass die meisten Mitarbeitenden nur `0-20%` ihrer Arbeit vollständig delegieren würden.
- Die ASE-2025-Studie zu Developer-Agent-Collaboration fand mehr Erfolg bei inkrementeller Zusammenarbeit als bei One-shot-Delegation.
- Aktive Iteration, Debugging und Testing mit dem Agenten blieben zentrale Erfolgsfaktoren.

Neue Schwerpunktkompetenzen

Wichtiger als früher

Kompetenz	Warum
Konzeptverständnis	AI beschleunigt Implementierung, nicht die Systemidee
Problemzerlegung	Gute Delegation setzt gute Problemformulierung voraus
Review & Supervision	AI-Ausgaben müssen verifiziert und eingeordnet werden
Kommunikation	Anforderungen zwischen Menschen, Agenten und Systemen

Unwichtiger als früher

- jedes Detail eines Nischentools auswendig kennen
- Boilerplate selbst schreiben
- Implementierungswissen mit Engineering-Verantwortung verwechseln

> Je mehr Coding delegiert wird, desto wichtiger werden **Architektururteil**, **Qualitätsmaßstäbe** und **verantwortliche Aufsicht**.

Vorsicht: Das ist keine vollständige Ersetzung

- Die Studien belegen Produktivitäts- und Delegationsgewinne, aber nicht das Ende menschlicher Softwareentwicklung.
- Die stärksten Aussagen zu "full-stack" und Skill-Verschiebung stammen teilweise von frühen AI-Adoptern; sie sind richtungsweisend, aber nicht automatisch repräsentativ für alle Teams.
- Eine belastbare Schlussfolgerung ist trotzdem: Je mehr Coding delegiert wird, desto wichtiger werden Architektururteil, Qualitätsmaßstäbe und verantwortliche Aufsicht.

> Im Zeitalter von AI verschiebt sich Web Engineering von primär manueller Implementierung hin zu Systemdenken, Delegation, Bewertung und Verantwortung. Gerade deshalb wird Web Engineering wichtiger, nicht weniger.

Referenzen

- Microsoft Research, June 2025: *The Effects of Generative AI on High-Skilled Work*
- Anthropic, April 28, 2025: *Anthropic Economic Index: AI's impact on software development*
- ASE 2025: *Why AI Agents Still Need You*
- Anthropic, December 2, 2025: *How AI is transforming work at Anthropic*

Web-Systeme und Daten

Leitfrage: Warum gehört in dieser Veranstaltung zur Web-Perspektive ausdrücklich auch eine Datenperspektive?

Ein konkretes Beispiel: Der personalisierte Feed

Was der Nutzer sieht: eine personalisierte Startseite mit Empfehlungen - scheinbar einfach.

Vereinfachter Fluss

1. Der Nutzer sendet einen HTTP-Request.
2. Ein Load Balancer oder API Gateway leitet den Request weiter.
3. Der Recommendation Service liest Features aus einer Data Pipeline.
4. Ein ML-Modell erzeugt ein Ranking.
    - Das ML-Modell muss vorab trainiert werden
    - Das Training muss auf neuen Daten regelmäßig wiederholt werden
5. Der Service liefert die Empfehlungen über CDN und Frontend aus.

Web und Daten sind zwei Seiten derselben Medaille - ohne Datenpipeline kein Feed, ohne skalierbare Web-Architektur keine Auslieferung.

Vom Web zum Data Engineering

Warum Data Engineering dazugehört

- Web-Systeme erzeugen und konsumieren große Datenmengen
- Daten müssen gespeichert, verarbeitet und bereitgestellt werden
- moderne Anwendungen kombinieren APIs, Persistenz und Analyse

> **Konsequenz:**
>
> - viele Produktfunktionen hängen heute direkt von Datenqualität und Pipeline-Design ab
> - damit wird Data Engineering zum integralen Teil moderner Web-Systeme

Leitfragen für die nächsten Kapitel

Kapitel	Frage
2	Wie ist die Anwendung durch Architektur und Schichten strukturiert?
3	Welche technischen Grundlagen liefern HTTP, HTML und CSS?
4	Wie werden Web-Systeme serverseitig umgesetzt?
5	Wie werden Web-Prinzipien auf APIs übertragen?
6	Wie werden Datenflüsse und Datenprodukte systematisch betrieben?

Takeaway

Die Motivation der Veranstaltung ist doppelt: Web-Systeme sind technisch und organisatorisch komplex, und ihr Wert entsteht immer häufiger aus dem Zusammenspiel mit Daten. Dieser Kurs behandelt beides als zusammenhängende Disziplin.